Laboration 2: Cellular Automata

Group Supermodels
Vootele Mets
Juan Rodriguez
Marco Malosti
Björk Lucas
Sofia Fernandes
David Weingut

May 4, 2026

UPPSALA
UNIVERSITET

Contents

1 Problem statement **2**

2 Theoretical Background **3**
2.1 Game of Life and Logic Gates . . . 3
2.1.1 Game of Life as a discrete dynamical system . . . 3
2.1.2 Logic gates in the Game of Life . . . 4
2.1.3 Encoding of Inputs . . . 5
2.1.4 Reading outputs . . . 5
2.1.5 Initial conditions for the different gates . . . 5
2.2 Neuron Automaton Theory . . . 6
2.3 GHCA theory . . . 7

3 Model and parameters **8**

4 Numerical method **9**
4.1 Game of Life Method . . . 9
4.2 Neuron Automaton Method . . . 9
4.3 GHCA Method . . . 9

5 Results **10**
5.1 Game of Life Results . . . 10
5.2 Neuron Automaton Results . . . 11
5.3 GHCA Results . . . 13

6 Interpretation **14**

7 Conclusion **15**

8 Use of LLM tools **15**

1. Problem statement

In this lab we will be investigating classical examples of cellular automata. Cellular automata are ...

The goal of the lab is to ...

All of these tasks will give us an overview of the characteristics in cellular automata ...

2. Theoretical Background

In this section we present the theory needed to solve the tasks of the lab.

2.1. Game of Life and Logic Gates

2.1.1. Game of Life as a discrete dynamical system

Conway's Game of Life can be viewed as a discrete dynamical system on a rectangular lattice $X^{(t)} \in \{0, 1\}^{m \times n}$, where each entry $X_{ij}^{(t)}$ represents the state of the cell at site (i, j) at time t. The evolution of the system is determined by update rules that specify the state of each cell at the next time step, $X_{ij}^{(t+1)}$.

Definition 2.1 – Conway's rules [2]

1. **Survival**: a live cell stays alive if it has 2 or 3 live neighbors.
2. **Birth**: a dead cell becomes alive if it has exactly 3 live neighbors.
3. **Death by isolation**: a live cell dies if it has fewer than 2 live neighbors.
4. **Death by overcrowding**: a live cell dies if it has more than 3 live neighbors.
5. **Neighborhood**: the neighbors of a cell are the 8 surrounding cells.

Let $N_{ij}^{(t)}$ denote the degree of node $X_{ij}^{(t)}$ at time t. Since each cell has eight neighbors, the relevant directions are the cardinal and intercardinal directions. Self-interaction is excluded, so the evolution of a cell depends only on its neighborhood. The quantity $N_{ij}^{(t)}$ can be computed as

$$N_{ij}^{(t)} = \sum_{p=-1}^{1} \sum_{q=-1}^{1} K_{p+2,q+2} X_{i+p,j+q}^{(t)}, \tag{2.1}$$

where K is a 3×3 kernel. Equivalently,

$$N^{(t)} = K * X^{(t)}, \tag{2.2}$$

where $K * X^{(t)}$ denotes the convolution of the lattice $X^{(t)}$ with the kernel K.

The update rule can then be written as

$$X_{ij}^{(t+1)} = \begin{cases} 1, & \text{if } X_{ij}^{(t)} = 1 \text{ and } N_{ij}^{(t)} \in \{2, 3\}, \\ 1, & \text{if } X_{ij}^{(t)} = 0 \text{ and } N_{ij}^{(t)} = 3, \\ 0, & \text{otherwise.} \end{cases} \tag{2.3}$$

In vectorized form, this becomes

$$X^{(t+1)} = \mathbb{1}\left(N^{(t)} = 3\right) + X^{(t)} \odot \mathbb{1}\left(N^{(t)} = 2\right), \tag{2.4}$$

and we use dirichlet boundary conditons or zero padding to handle the adjacency of boundary nodes.

2.1.2. Logic gates in the Game of Life

In classical computation, basic logic gates such as AND, OR, and NOT act on one or two input streams and produce a single output stream. For example,

$$\begin{array}{c} \texttt{01001010} \\ \underline{\texttt{10101010}} \\ \texttt{00001010} \\ \texttt{AND} \end{array} \qquad \begin{array}{c} \texttt{01001010} \\ \underline{\texttt{10101010}} \\ \texttt{11101010} \\ \texttt{OR} \end{array} \qquad \begin{array}{c} \underline{\texttt{01001010}} \\ \texttt{10110101} \\ \\ \texttt{NOT} \end{array}$$

Such gates implement a local transformation of binary information.

In the Game of Life, which is a discrete dynamical system evolving on a lattice, one may view logic gates from a different perspective: not as abstract Boolean operators, but as spatially embedded computational devices. The initial state is a geometric configuration, and the update rule propagates that configuration through time (changes its geometry from one to another). From this viewpoint, the construction of a logic gate becomes a problem of geometric constraint and synchronization. The central question is which initial lattice configuration (i.e., which geometry) $X^{(0)}$ can, under Conway's rules, evolve so as to realize deterministic, time-dependent binary information processing in the form of `AND`-, `OR`-, and `NOT`-gates?

First, we need to construct an initial geometry that evolves, under Conway's rules, into a space-time signal. For this we could use a train of *gliders* that travels periodically and diagonally across the lattice [3]. To get an arbitrary length of space-time signals we migh use *guns* that generate gliders [4]. The trajectories along which gliders evolve act as rigid wiring, thereby constraining the direction of information flow. In the absence of any interference, the signal propagates persistently along these paths. To obtain a richer computational structure, this simple stream of gliders (interpretable as a sequence of 1s) must be complemented with the ability to represent absence (0s) as meaningful information. Furthermore, one requires mechanisms for interaction between multiple input streams such that they can be combined to produce a third output stream, with the resulting configurations remaining composable within larger constructions.

It is possible to realize [1] composable logic gates in the Game of Life by spatially arranging glider guns, eaters, and collision configurations such that controlled streams of gliders encode binary information. In this setting, glider presence corresponds to logical `1`, while the absence of a glider stream corresponds to `0`. By appropriately translating and positioning glider guns, and by placing eaters as stabilizing and filtering structures, one can engineer deterministic collision patterns that implement logical operations. Moreover, these constructions are required to be composable, in the sense that the output of one gate can be routed as the input to another without loss of computational fidelity.

2.1.3. Encoding of Inputs

See `GoLogic.py` and the different input lanes, maybe put images of the gun, eater and glider i don't know.

2.1.4. Reading outputs

Reading outputs should be over a 4×4 bounding box, where a glider fits in? The glider in the output has an intercardinal direction. Synchronization?

2.1.5. Initial conditions for the different gates

See `GoLogic.py` for the different gates.

2.2. Neuron Automaton Theory

FOR THEORY OF GHCA...

2.3. GHCA theory

GHCA THEORY HERE

3. Model and parameters

MODEL AND PARAMETERS?

4. Numerical method

In this section we will describe the numerical procedures used in our code and state the relevant parameters to reproduce our findings.

4.1. Game of Life Method

`scipy convolve` acording to theory uses `C` for computations.

8-bit unsigned integers `np.uint8` to store 0s and 1s and decrease uncesseracy memory usage.

Parse RLE encoded strings for basic components (gosper gun and eater).

Precomputation of states was employed so that the `GameOfLifeDebug.py` script was able to step backwards in time, since the update has no inverse. Helpful for validation.

4.2. Neuron Automaton Method

FOR IMPLEMENTATION OF NEURON AUTOMATON...

4.3. GHCA Method

FOR IMPLEMENTATION OF GHCA...

5. Results

5.1. Game of Life Results

FOR RESULTS OF Game of Life...

5.2. Neuron Automaton Results

Using random initial conditions in the neuron automation usually led to either the whole system eventually dying out and being inert in the ”ready” condition or to infinitely forward moving gliders appearing. By experimenting several initial conditions were constructed that would lead to interesting developments of the system such as a periodic oscillator or a pattern that infinitely generates more moving patterns. Some interesting initial conditions are pictured below.

First a number of infinitely moving gliders were identified. On figure 1 are brought three examples. Those kinds of patterns move infinitely in the initial direction unless they collide with another pattern.

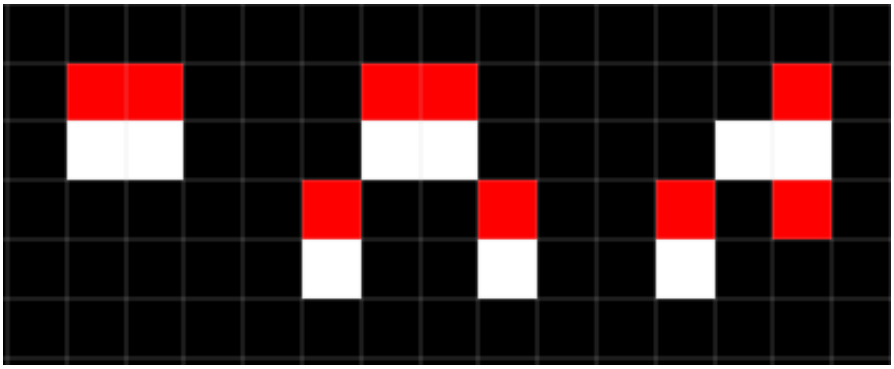

Figure 1. Three examples of infinitely moving patterns. Two patterns on the left move in the upward direction and the one on the right moves diagonally to right-left.

By having two simple gliders collide with eachother using initial conditions shown in figure 2 it is possible to create two new gliders that move at a right angle to the initial motion giving the impression of two bouncing gliders.

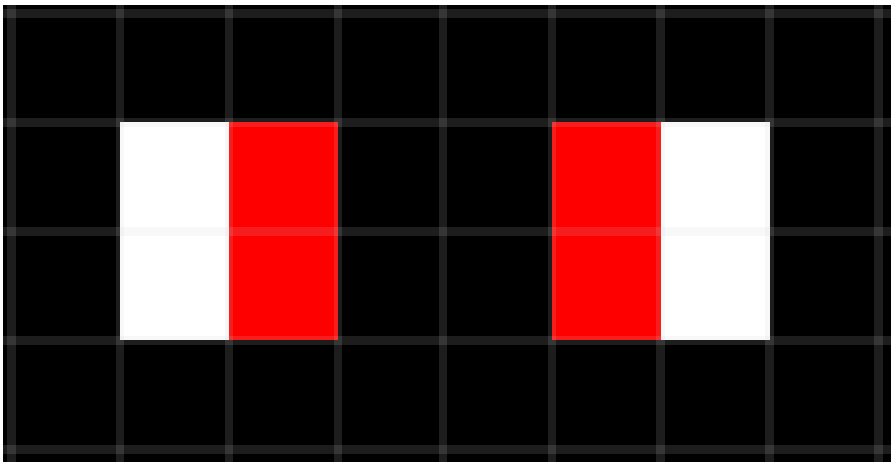

Figure 2. Initial condition that leads to two gliders bouncing in a 90 degree angle

It is also possible to set up a pattern that stais in the same location and oscillates infinitely. An oscillator with three distinct steps was identified that is pictured in figure 3

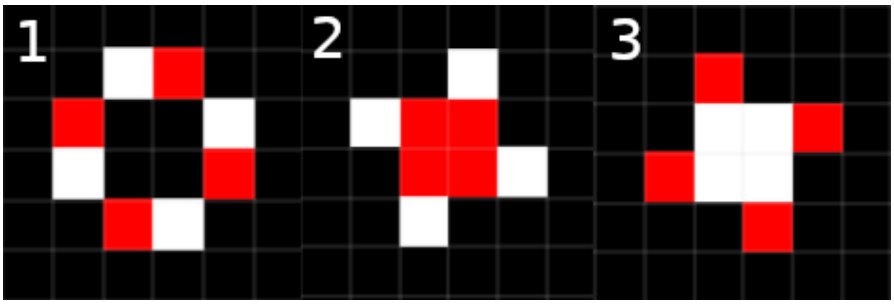

Figure 3. A three step oscillating pattern

5.3. GHCA Results

FOR RESULTS OF GHCA...

6. Interpretation

INTERPRETATION GOES HERE.

7. Conclusion

CONCLUSION GOES HERE

8. Use of LLM tools

USAGE OF LLM GOES HERE.

Files submitted

The following files have been submitted with this report:

1. `common.py`
2. `periodicPoints.py`
3. `phasePortrait.py`
4. `bifurcationCascade.py`

References

[1] Alex Bellos. Game of life: Logic gates.

[2] Jordi-Lluís Figueras. Interactive conway's game of life board for classroom demonstrations, 2026.

[3] LifeWiki. Glider.

[4] LifeWiki. Glider gun.